Pilot: AI Project Coordinator (PRD v1.0)

Kurukshetra Hackathon · Agentic AI track · W-PS-1 (sponsor: Worcflow) · 24 hours · 4 builders

0. How to read this document

This is the single source of truth for the team and for any AI coding agent working in the repo. Table names, status values, endpoint paths and JSON field names are exact: do not rename them. If something is unclear, pick the simplest option that satisfies the acceptance checklist (section 20) and note the decision in `/docs/decisions.md`. Working title: **Pilot**.

1. Summary

Pilot is a Telegram bot + GitHub integration + Jira-style web app that runs a college group project. A lead describes the project in Telegram. Pilot creates the GitHub repo, invites teammates, generates detailed tasks, bundles dependent tasks into independent blocks, and assigns blocks evenly. Then it runs a follow-up loop: it asks owners for updates on Telegram, interprets messy replies, and updates task status. When someone says done, Pilot checks their real commits on GitHub and returns a verdict with confidence. A **Guide agent** oversees everything: it knows who contributed what and when, catches contradictions between teammates, routes blockers to the person who can fix them, and shows the ripple effect of delays.

Pitch line: Other bots collect status updates. Ours checks them.

2. What the judges score (15 points)

Criterion	What judges check	Where we win
Functional Completeness (5)	task → follow-up → reply → status update, over a real messaging platform	Core loop (F8) works flawlessly, live
Quality of Execution (5)	accuracy, reliability, unexpected scenarios	Messy-reply handling, GitHub verification with confidence, retries
Thought, Effort, Creativity (5)	originality and depth	Verification, contradiction detection, blocker routing, ripple effect, Guide

Rules to respect: live demo is mandatory; messages and replies must pass through the real Telegram integration; canned messages alone do not count; judges may inject unexpected scenarios; depth beats feature count.

3. Users and roles

- **Lead:** creates the project, edits tasks, approves splits, can override verification.
- **Member:** joins via link, receives block assignments, replies to check-ins.
- **Guide:** the autonomous agent (backend service) that monitors and acts.
- **Judge:** may message the bot with unexpected input and ask the Guide questions.

4. Scope

In scope: Telegram bot, GitHub repo creation and collaborator invites, LLM task generation, dependency bundling, equal assignment, Jira-style web board, scheduled check-ins, reply interpretation, GitHub verification, contradiction detection, blocker routing, ripple effect, contribution ledger and charts, Guide Q&A and reports.

Out of scope: agile or sprint framework generation, native mobile app, browser extension, GitHub webhooks (we poll), user accounts and login (identity comes from Telegram; web app selects a project by code).

5. Architecture

Telegram users <---> Telegram Bot (python-telegram-bot, long polling)

v

```
React web app  <--->  FastAPI backend  <--->  SQLite
(polls every 2s)      |  |  |
|  |  +--> Scheduler (APScheduler): check-ins, GitHub poll, Guide tick
|  +------> LLM layer (llm.py): JSON-validated calls to OpenCode free models
+----------> GitHub layer (gh.py): repos, invites, commits, PRs, diffs
Guide agent (guide.py): tool-calling loop over backend functions
```

Bot and API run in **one Python process** sharing one database, so there is no service to sync. Telegram uses long polling and GitHub uses polling, so no public URL is needed.

6. Tech stack

Layer	Choice
Backend	Python 3.11, FastAPI, SQLModel + SQLite
Telegram	python-telegram-bot (v20+, async, long polling)
LLM	OpenCode free models (check current list in their docs), wrapped in one llm.py; Pydantic validation, JSON-only prompts, 2 retries
GitHub	PyGithub or raw REST with a personal access token (scopes: repo)
Scheduler	APScheduler (AsyncIOScheduler)
Graph logic	NetworkX or plain union-find
Frontend	React + Vite + Tailwind, dnd-kit (board), React Flow (graph), Recharts (contribution lines)
Live updates	Frontend polls /api/projects/{code}/snapshot every 2 seconds

7. Repo structure

```
/backend   main.py            # FastAPI app + starts bot and scheduler   db.py models.py    # SQLModel tables
(section 8)   llm.py            # call_json(), prompts   gh.py             # GitHub functions   planner.py
# task generation, bundling, assignment, deadlines   loop.py            # check-ins, reply handling, status
transitions   verify.py          # GitHub verification   guide.py           # Guide agent, contradiction,
blocker routing, ripple   bot.py             # Telegram handlers   routes.py          # REST endpoints (section
13) /frontend            # React app (section 15) /docs                # decisions.md, demo-evidence/
README.md  .env.example
```

8. Data model (SQLite)

```
project(id PK, code UNIQUE, name, idea, repo_full_name, repo_url, lead_member_id,         telegram_group_id,
status DEFAULT 'planning', starts_at, ends_at, last_polled_at); member(id PK, project_id, name, telegram_id,
github_login, skills, role DEFAULT 'member',        invite_status DEFAULT 'pending'); block(id PK, project_id,
name, total_effort, owner_member_id, oversize_flag DEFAULT 0); task(id PK, project_id, key,            -- key
like T-1      title, description, acceptance_criteria,  -- JSON list of strings      effort,
-- integer 1..8      status DEFAULT 'todo', is_milestone DEFAULT 0, block_id, owner_member_id,      deadline,
started_at, completed_at, progress DEFAULT 0, risk DEFAULT 'ok',      last_checkin_at, checkin_count DEFAULT 0);
 task_dep(task_id, depends_on_task_id, kind DEFAULT 'hard');   -- kind: hard | handoff message(id PK, project_id,
member_id, task_id, direction,     -- out | in         text, parsed_json, created_at); blocker(id PK, task_id,
description, type, needed_from, routed_to_member_id,         status DEFAULT 'open', created_at, resolved_at);
-- open | routed | resolved contradiction(id PK, project_id, task_ids, member_ids, summary,               status
DEFAULT 'open', created_at, resolved_at, resolution); commit_log(sha PK, project_id, author_login, member_id,
message, files, additions,           deletions, committed_at, task_id); verification(id PK, task_id, verdict,
confidence, matched_shas, criteria_results,              reason, created_at); event(id PK, project_id, ts,
actor_member_id, type, task_id, payload);
```

The event table is both the Jira-style activity log and the contribution ledger. Event types: project_created, member_joined, task_created, task_updated, tasks_bundled, block_assigned, checkin_sent, reply_received, status_changed, blocker_opened, blocker_routed, blocker_resolved, contradiction_opened, contradiction_resolved, commit_seen, verification_done, risk_changed, guide_note. **Every state change writes an event.**

9. Task status state machine

Statuses: todo, in_progress, blocked, claimed_done, needs_review, verified_done.

- todo → in_progress: first progress reply, or first matching commit.
- in_progress ↔ blocked: blocker reply opens a blocker; resolving it returns the task to in_progress.
- any open status → claimed_done: member claims completion.
- claimed_done → verified_done: verifier verdict is verified with confidence ≥ 0.7.
- claimed_done → needs_review: verdict is partial, or no_evidence, or confidence below 0.7.
- needs_review → claimed_done: member reports more work or a new matching commit appears (re-verify).
- needs_review → verified_done: lead override (button in web app).
- verified_done → in_progress: lead reopens.

The bot never sets verified_done from a chat reply alone. Only the verifier or a lead override can.

10. End-to-end workflow, A to Z

Step 1. Pre-event setup. Create the bot with @BotFather and save the token. Create a GitHub personal access token (repo scope) for the account that will own demo repos. Fill .env. Start backend, bot and frontend. All four builders confirm they can message the bot.

Step 2. Project creation (Telegram). The lead sends `/newproject <idea text>`. The backend calls the LLM (P1) to produce name, one-paragraph summary, and duration in hours (default 20 if missing). It inserts a project with a short code, creates the lead as a member (role lead), and calls GitHub to create repo `pilot-<slug>-<code>` with a README and a CONTRIBUTING.md stating the commit rule: **every commit message must contain its task key, like [T-4]**. The bot replies with the repo URL, the project code, and a join link `https://t.me/<bot>?start=join_<code>`.

Step 3. Members join. Each member opens the join link. The bot receives `/start join_<code>`, records telegram_id and name, and asks for a GitHub username. The backend validates it (GET /users/{login}), sends a collaborator invite (PUT /repos/{owner}/{repo}/collaborators/{login}), stores github_login, and tells the member to accept the invitation on GitHub. The poller marks invite_status = accepted when the user appears in the collaborators list. Event: member_joined.

Step 4. Task generation. The lead sends `/plan` (or clicks Generate Tasks in the web app). The LLM (P2) receives the idea, team size, skills list and duration, and returns 12 to 20 detailed tasks with description, 2 to 4 acceptance criteria, effort 1 to 8, dependencies by title, and milestone flags (including a final Demo milestone). The planner validates the result: unique titles, dependency titles exist, **cycles removed** (drop the offending edge), at least one milestone. Keys T-1..T-n are assigned. Events: task_created.

Step 5. Human edit on the web app. The lead opens the Tasks page: edit any field, add a task, delete a task, edit dependencies. Delete removes the task and its edges. All edits write task_updated events.

Step 6. Bundling. The lead clicks Bundle & Assign. The planner builds a graph of non-milestone tasks with hard dependency edges and computes connected components (section 11, F5). Each component becomes one block: dependent tasks are bundled into a single independent unit of work. Milestone tasks are excluded from bundling and depend on blocks.

Step 7. Equal assignment. The planner assigns blocks to members with greedy load balancing (F6). Oversize blocks are flagged. The lead sees a preview (blocks, owners, loads) and confirms. Events: tasks_bundled, block_assigned.

Step 8. Deadlines and kickoff. Deadlines are computed from the project window and dependency order (F4). The lead sends `/kickoff` or clicks Start Project. Every member receives a Telegram message listing their block, each task with deadline and acceptance criteria, and the commit rule. The scheduler is armed.

Step 9. Core loop: check-in. Every 60 seconds the scheduler selects tasks needing a check-in (F8 rules) and sends the owner a Telegram message asking for a specific update on a specific task. Event: checkin_sent. Message stored with direction = out.

Step 10. Core loop: reply interpretation. Any incoming Telegram text goes to the reply handler. It determines which task the reply is about, sends the reply plus context to the LLM (P3), and gets structured JSON: intent, new status, blocker, progress, evidence links, claims about others, clarification need. Low confidence or unclear replies trigger one clarifying question instead of a status change.

Step 11. Status update and side effects. The handler applies the transition (section 9), writes events, and then runs side effects in order: (a) if a blocker is present, open it and run blocker routing (F12); (b) run contradiction check (F11); (c) recompute ripple risk for downstream tasks (F13); (d) if the intent is completion, go to step 12.

Step 12. Completion claim triggers GitHub verification. Status becomes claimed_done. The verifier (F9) gathers the member's commits and PRs since the task started, summarizes diffs, and asks the LLM (P4) to judge them against the acceptance criteria.

Step 13. Verdict handling. Verified (≥ 0.7): verified_done, member congratulated, completed_at set, downstream tasks recalculated. Partial or no_evidence: needs_review, the bot tells the member exactly what is missing (for example criterion 2 has no matching code, or no commits with [T-4] found) and invites them to push and reply again. The web board shows both the claim and the verdict.

Step 14. Guide oversight. Every 5 minutes the Guide scans the project: overdue tasks, stale in_progress tasks with no commits, blocked tasks with no routing, unresolved contradictions. It takes actions (nudge, escalate, route) and writes guide_note events. It also answers questions from Telegram `/ask` and the web chat panel.

Step 15. Live visualization. The web app polls the snapshot endpoint and updates board, graph, activity log, and contribution chart without refresh.

Step 16. Wrap-up. `/report` or the Report button makes the Guide produce a final summary: who contributed what (commits, lines, tasks verified), what was completed and when, claimed versus verified counts, blockers and how long they lasted, and unresolved risks.

11. Features and how to implement each

F1. Telegram pipeline (owner A)

Handlers: `/newproject`, `/start join_<code>`, `/plan`, `/kickoff`, `/mytasks`, `/status`, `/ask <question>`, `/report`, `/help`, and a catch-all text handler for replies. Identity: look up member by telegram_id. Free text from a member goes to the reply handler (F8). Keep per-member conversation state in memory and DB: active_task_id (last task asked about) and awaiting_clarification flag. Always write inbound and outbound text to message. Wrap every handler in try/except and reply with a friendly error so the demo never goes silent.

F2. GitHub pipeline (owner B)

Functions in gh.py: create_repo(name), add_collaborator(repo, login), list_collaborators(repo), list_commits(repo, since), get_commit(repo, sha) (files, additions, deletions, patch), list_prs(repo). Poller (every 45 seconds): fetch commits since project.last_polled_at, upsert into commit_log, map author_login to member_id, extract task key with a regex for \[T-\<number>\] (else leave task_id null and let the verifier match semantically), write commit_seen events, move todo → in_progress for the matching task. Cache everything; GitHub rate limit is 5000 requests per hour with a token, which is plenty at this volume.

F3. LLM layer (owner C)

call_json(system, user, model_cls, retries=2): temperature 0.1, instruct JSON-only output, strip code fences, validate with Pydantic, on failure retry with the validation error appended. Log every prompt and response to /docs/demo-evidence/llm.log. Fallback: if all retries fail, return a safe default (intent unclear, ask a clarifying question). Keep model name in config so it can be swapped.

F4. Task generation and deadlines (owner C)

LLM P2 produces tasks. Deadlines: take the project window [starts_at, ends_at], reserve the last 10 percent for the final milestone, and for each block distribute its tasks in topological order, each task's duration proportional to its effort, so a task's deadline is block_start + cumulative_effort / block_total_effort * block_window. Milestone deadline equals ends_at. In demo mode, a config switch compresses deadlines into minutes so check-ins fire live.

F5. Bundling dependent tasks into independent blocks (owner C)

```
def bundle(tasks, deps):          # non-milestone tasks, hard deps only     parent = {t.id: t.id for t in tasks}
     def find(x):         while parent[x] != x:             parent[x] = parent[parent[x]]; x = parent[x]
return x     for a, b in deps:         parent[find(a)] = find(b)     blocks = {}     for t in tasks:
blocks.setdefault(find(t.id), []).append(t)     return list(blocks.values())     # each list is one independent
block
```

Block weight is the sum of task effort. Each task row gets block_id. A block can be worked end to end by one person without waiting on anyone else.

F6. Equal assignment (owner C)

Sort blocks by weight descending. For each block, assign to the member with the smallest current load (sum of assigned weights); ties broken by skill match (overlap between block text and member.skills). Compute the fair share total /

members. If a block exceeds 1.4 × fair share, set oversize_flag and let the Guide propose a split point: the task boundary inside the block (in topological order) that best balances load. If the lead approves, the dependency across the boundary changes from hard to handoff, the block splits in two, and assignment reruns. Handoff edges are the only cross-person dependencies, and the ripple feature tracks them.

F7. Jira-style web app (owner D)

See section 15. Key behavior: board columns follow the status machine, cards show owner color, deadline, risk badge, and a verification badge; clicking a card opens a drawer with description, criteria, dependencies, blocker, activity, verification report, and message thread. The Plan page allows add, edit, delete, regenerate, bundle, assign, kickoff.

F8. Core loop: check-ins and replies (owner A)

Check-in selection (scheduler every 60 seconds) for tasks with status in todo, in_progress, blocked and project status running:

- Kickoff message once at start.
- Cadence: every CHECKIN_HOURS (default 4) since last_checkin_at.
- Extra pings at deadline minus 24h, deadline minus 2h, and when overdue.
- Cooldown: at most one check-in per member per 30 minutes (bundle several tasks into one message); quiet hours 22:00 to 08:00 skipped.
- Escalation: after 2 unanswered check-ins, notify the lead via the Guide.
- Demo override: the Run follow-up now button and the `/followup` command ignore cadence and cooldown.

Check-in text is generated by the LLM or a template with task title, deadline, and a gentle question, plus inline buttons: In progress, Blocked, Done. Buttons are a convenience; free text must work equally well.

Reply routing: if active_task_id exists and the reply is plausibly about it, use it; otherwise pass all of the member's open tasks to the LLM and let it pick task_key. Replies about several tasks in one message are allowed.

Handling matrix:

- progress_update → set in_progress, store progress percent.
- blocker → open blocker, set blocked.
- completion_claim → claimed_done, trigger verification.
- question → answer from task data or hand to the Guide.
- off_topic or unclear → one clarifying question, no status change.
- Silence → covered by escalation above.

F9. GitHub verification (owner B)

1. Deterministic pre-check: if the member has zero commits since task.started_at (or task creation), return no_evidence without calling the LLM.
2. Collect candidate commits: those with the [T-key] first, then the member's other commits in the window. Fetch files, additions, deletions, and truncated patches (cap 6000 characters total).
3. Call P4 with title, description, acceptance criteria, and commit summaries.
4. Return {verdict, confidence, matched_shas, criteria_results[{criterion, met, note}], reason}; store in verification.
5. Rules: verified and confidence ≥ 0.7 → verified_done. Otherwise needs_review. If the matching commits were authored by someone else, say so and credit them in the ledger.
6. Re-verification triggers when a new commit with the task key arrives while the task is needs_review, or when the member replies done again.

The UI always shows confidence and reason, never a bare pass or fail.

F10. Guide agent (owner C)

A tool-calling loop (max 6 steps per run). **Read tools:** get_project_snapshot(), get_task(key), get_member_contributions(member), get_commits(task_or_member), get_timeline(until), get_open_blockers(), get_risks(). **Action tools:** message_member(member, text), flag_contradiction(...), route_blocker(blocker_id), propose_split(block_id), post_note(text). If the model has no native tool calling, use a JSON action loop: the model returns a tool name and args, the backend executes and feeds back the result, until the model returns a final text.

Modes: (1) **Proactive tick** every 5 minutes with a checklist prompt (overdue, stale, unrouted blockers, open contradictions). (2) **Conversational** via `/ask` and the web chat, answering things like who contributed what, what was completed by a given time, and why a milestone is at risk. (3) **Reports**: daily digest and final report. Guide answers must cite data (task keys, commit SHAs, timestamps) from tools, never invent.

Contribution ledger: per member, count commits, lines added and removed, tasks verified, tasks claimed, blockers resolved for others. Timeline: first and last commit times, completed_at per task.

F11. Contradiction detection (owner C)

Two layers.

- **Rules (cheap, run on every status change):** (1) member says a dependency or another person's work is done, while that task is not verified_done or its owner reported blocked; (2) two members describe the same shared item with opposite status; (3) claimed_done contradicted by verification no_evidence.
- **LLM (P5):** feed the latest messages and statuses of related tasks (same block, handoff neighbors, milestone predecessors) and ask for conflicts.

On detection: insert contradiction, message both people with the specific conflict and one question each (for example: Meera said the integration is done, Kabir said he is waiting on the API. Which is accurate?), and mark resolved when replies agree or GitHub evidence settles it. Show open contradictions as a red banner on the board.

F12. Blocker routing (owner C with A)

When a reply yields a blocker with description, type and needed_from, the Guide picks who can unblock it. Candidate order: (1) owner of the task named in needed_from or the upstream task; (2) member whose skills match; (3) the lead. Call P6 with the member list, owners, and skills to choose one with a reason. Message the candidate: Kabir is blocked on test credentials for T-7. Can you help? with buttons I can help and Not me. Set blocker.status = routed. If no answer in 2 hours (demo: 2 minutes) move to the next candidate and finally notify the lead. When the helper confirms or the blocked member says it is resolved, set blocker resolved, notify the owner, return the task to in_progress. Log every hop as an event.

F13. Ripple effect (owner C data, D visuals)

Tasks form a DAG with hard edges, handoff edges, and milestone edges. For each task compute slip, how far its projected finish is behind its deadline: for a blocked or overdue task, slip = now - deadline (or time since blocked, whichever is larger); for downstream tasks, propagate slip along edges. A downstream task has slack = deadline - upstream_deadline - effort_hours. Risk: critical if slip > slack and the task is within 12 hours of its deadline or already late; at_risk if slip > 0.5 × slack; else ok. Milestone risk is the worst risk among its predecessors. Store task.risk; write risk_changed events. Frontend shows the graph with edges from the blocked node glowing red and a text line: Kabir is blocked, so the Demo milestone is at risk.

F14. Contribution visuals (owner D)

Series per member from commit_log (cumulative commits or lines per hour, one color per member) in a Recharts line chart, plus a table: member, commits, lines, tasks verified, last active. The same colors identify owners everywhere (board, graph nodes, chart).

12. Configuration & Environment

`.env` (see `.env.example`):

```
TELEGRAM_BOT_TOKEN= GITHUB_TOKEN=                  # classic PAT with repo scope GITHUB_OWNER=                  # account that owns demo repos LLM_BASE_URL=                  # OpenCode / OpenRouter compatible endpoint LLM_API_KEY= LLM_MODEL=                       # e.g. qwen/qwen3-coder:free or current free model DATABASE_URL=sqlite:///./pilot.db CHECKIN_HOURS=4 DEMO_MODE=false               # when true, compresses deadlines to minutes QUIET_HOURS_START=22 QUIET_HOURS_END=8 FRONTEND_ORIGIN=http://localhost:5173
```

All secrets stay in `.env`. Never commit them. The backend loads them once at startup and exposes a read-only `/api/health` that reports which integrations are alive (Telegram, GitHub, LLM).

Demo mode switch: when `DEMO_MODE=true`, project duration is forced to ~30–45 minutes, check-in cadence drops to 2–3 minutes, quiet hours are ignored, and blocker escalation timeout becomes 2 minutes. This is the only way the live demo can show the full loop without waiting hours.

13. REST API endpoints

All routes live under `/api`. Project is always identified by its short `code` (not numeric id) so the web app never needs an auth token.

Health & meta

Method	Path	Purpose
GET	`/api/health`	{telegram, github, llm, db} status
GET	`/api/projects`	list recent projects (code, name, status)

Project lifecycle

Method	Path	Purpose
POST	`/api/projects`	create (internal, normally from bot)
GET	`/api/projects/{code}`	basic project metadata
GET	`/api/projects/{code}/snapshot`	**full live state** (polled every 2 s)
POST	`/api/projects/{code}/plan`	trigger LLM task generation
POST	`/api/projects/{code}/bundle`	run bundling + assignment, return preview
POST	`/api/projects/{code}/confirm-assignment`	lock the assignment
POST	`/api/projects/{code}/kickoff`	start project, arm scheduler, send kickoff messages
POST	`/api/projects/{code}/followup`	force one check-in cycle (demo)
POST	`/api/projects/{code}/report`	ask Guide for final report

Tasks

Method	Path	Purpose
GET	`/api/projects/{code}/tasks`	list tasks
POST	`/api/projects/{code}/tasks`	add task
PATCH	`/api/projects/{code}/tasks/{key}`	edit any field (title, description, effort, criteria, deps, status override)
DELETE	`/api/projects/{code}/tasks/{key}`	delete task + edges
POST	`/api/projects/{code}/tasks/{key}/override-verify`	lead forces verified_done

Members, blockers, contradictions, events

Method	Path	Purpose
GET	`/api/projects/{code}/members`	list with invite_status
GET	`/api/projects/{code}/blockers`	open + recent
GET	`/api/projects/{code}/contradictions`	open + recent
GET	`/api/projects/{code}/events`	activity log (newest first, limit/offset)
GET	`/api/projects/{code}/contributions`	ledger + time series for charts
POST	`/api/projects/{code}/ask`	Guide Q&A (body: {question})

Snapshot shape (critical contract)

```
{   "project": { "code", "name", "status", "repo_url", "starts_at", "ends_at" },   "members": [ { "id", "name",
"role", "github_login", "invite_status", "color" } ],   "blocks": [ { "id", "name", "owner_member_id",
"total_effort", "oversize_flag" } ],   "tasks": [ {     "key", "title", "status", "owner_member_id", "block_id",
    "deadline", "progress", "risk", "is_milestone",     "acceptance_criteria", "description",
"verification": { "verdict", "confidence", "reason" } | null,     "blocker": { "id", "description", "status",
"routed_to" } | null   } ],   "deps": [ { "from": "T-3", "to": "T-1", "kind": "hard|handoff" } ],   "events":
[ /* last 30 */ ],   "contradictions": [ /* open */ ],   "contributions": {     "series": { "member_id":
[ { "ts", "commits", "lines" } ] },     "totals": [ { "member_id", "commits", "lines", "tasks_verified",
"last_active" } ]   },   "guide_notes": [ /* recent */ ] }
```

Any field name above is exact. Frontend must not invent aliases.

14. Scheduler jobs

APScheduler AsyncIOScheduler, started in `main.py` alongside the FastAPI lifespan and the Telegram bot.

Job	Interval	Action
checkin_tick	60 s	F8 selection + send
github_poll	45 s	F2 commit fetch + upsert
guide_tick	5 min	F10 proactive scan
invite_check	60 s	mark accepted collaborators
risk_recompute	2 min	F13 ripple

All jobs are idempotent and write events. In DEMO_MODE the intervals are shortened (checkin 20 s, guide 60 s, escalation 2 min).

15. Frontend (owner D)

Pages / routes

- `/` — project picker (enter code or list recent)
- `/p/{code}` — main board (Kanban)
- `/p/{code}/plan` — task editor + Bundle & Assign + Kickoff
- `/p/{code}/graph` — dependency / ripple graph (React Flow)
- `/p/{code}/contributions` — Recharts line chart + table
- `/p/{code}/activity` — full event log
- `/p/{code}/chat` — Guide chat panel

Board (Kanban)

Columns exactly match the status machine: Todo | In Progress | Blocked | Claimed Done | Needs Review | Verified Done.

Card content:

- task key + title
- owner avatar / color
- deadline (relative + absolute)
- risk badge (ok / at_risk / critical)
- verification badge (confidence % or “no evidence”)
- progress bar if in_progress

Click opens a right-hand drawer with:

- full description + acceptance criteria checklist
- dependency list
- current blocker (if any) + routing status
- verification report (reason + matched SHAs)
- message thread (in + out)
- lead-only “Override to Verified” button

Plan page

- editable table of tasks
- add / delete / regenerate
- dependency editor (simple multi-select or React Flow mini)
- “Bundle & Assign” button → preview modal (blocks, owners, loads, oversize warnings)
- “Confirm & Kickoff” button

Graph page

React Flow DAG. Nodes colored by owner. Edges colored by kind (hard = solid, handoff = dashed). When a task is blocked or critical, downstream edges glow red and a caption appears: “Kabir is blocked on T-7 → Demo milestone at risk”.

Contributions page

Recharts multi-series line chart (one color per member, same colors as board). Toggle commits vs lines. Table underneath with totals.

Live updates

Single `useSnapshot(code)` hook that polls `/api/projects/{code}/snapshot` every 2 s and replaces local state. No websockets required.

Visual language

- Owner colors are stable (hash of member_id → palette of 8 distinct colors).
- Risk: green / amber / red.
- Verification confidence shown as a small pill, never a bare pass/fail.

16. LLM prompts (P1–P6)

All prompts force pure JSON. `call_json` strips fences and validates against a Pydantic model. Temperature 0.1. Two retries with the validation error appended.

P1 – Project parse (`/newproject`)

System: “You extract a short project name, one-paragraph summary, and estimated duration in hours from a free-text idea. Reply only with JSON.”

User: the idea text.

Output schema: `{ "name": str, "summary": str, "duration_hours": int }`

P2 – Task generation (`/plan`)

System: “You are a project planner for a college hackathon team. Produce 12–20 concrete engineering tasks. Each task needs title, description, 2–4 acceptance criteria, effort 1–8, dependencies (list of other titles), and is_milestone flag. Include a final Demo milestone. No cycles. Reply only with JSON.”

User: idea + team size + skills + duration.

Output schema: `{ "tasks": [ { "title", "description", "acceptance_criteria": [str], "effort": int, "depends_on": [str], "is_milestone": bool } ] }`

P3 – Reply interpretation (core loop)

System: “You interpret a team member’s free-text reply about their work. Decide intent, which task, new status, progress %, blocker details, claims about other people, and whether clarification is needed. Reply only with JSON.”

User: member’s open tasks + recent messages + the new reply.

Output schema: `{ "task_key": str|null, "intent": "progress_update"|"blocker"|"completion_claim"|"question"|"off_topic"|"unclear", "new_status": str|null, "progress": int|null, "blocker": { "description", "type", "needed_from" }|null, "claims_about_others": [str], "needs_clarification": bool, "clarifying_question": str|null, "confidence": float }`

P4 – Verification (F9)

System: “You judge whether the supplied commits and patches satisfy the acceptance criteria of a task. Be strict. Reply only with JSON.”

User: task title + description + criteria + list of commit summaries (sha, message, files, truncated patch).

Output schema: `{ "verdict": "verified"|"partial"|"no_evidence", "confidence": float, "matched_shas": [str], "criteria_results": [ { "criterion": str, "met": bool, "note": str } ], "reason": str }`

P5 – Contradiction check

System: “Look for factual contradictions between recent status statements of teammates who share dependencies or a milestone. Reply only with JSON.”

User: latest messages + statuses of related tasks.

Output schema: `{ "has_contradiction": bool, "summary": str, "task_ids": [str], "member_ids": [int], "questions_to_ask": [str] }`

P6 – Blocker routing

System: “Given a blocker and the list of members with their skills and owned tasks, choose the single best person to ask for help and explain why. Reply only with JSON.”

User: blocker description + needed_from + member list.

Output schema: `{ "member_id": int, "reason": str }`

17. Error handling & resilience

- Every Telegram handler is wrapped in try/except; on failure the bot replies “Something went wrong, try again in a moment” and logs the traceback.
- LLM calls: 2 retries with validation error; final fallback is “unclear → ask clarifying question”.
- GitHub rate-limit or network errors: the poller skips the cycle and retries next interval; never crashes the process.
- SQLite: single writer (the Python process); no concurrent write risk.
- If repo creation fails, the bot still creates the project and tells the lead the GitHub step failed so they can paste a pre-made repo URL.
- All state changes are transactional (event + entity update in one commit).

18. Demo mode, seeding & cut order

Seeding script (`/backend/seed_demo.py`): creates a realistic project with 4 members, 14 tasks already bundled and assigned, a few commits, one open blocker, one contradiction, and a partially filled contribution chart. Used if live repo creation fails on stage.

Demo override buttons / commands (must work):

- Web: “Run follow-up now”
- Telegram: `/followup`
- Web: “Force verification on selected task”
- Web: “Inject sample reply” (for judges who want to type messy text)

Cut order if behind (from the locked plan):

7. Contribution chart visuals (keep the ledger data)
8. Ripple effect visuals (keep the risk numbers)
9. Guide Q&A chat panel (keep automated reports and proactive notes)
10. Auto repo creation (fall back to pre-made repo)

Never cut the core loop (check-in → reply → status → verification).

19. Team responsibilities & 24-hour timeline

Person	Owns
A	Telegram + LLM reply handling, scheduler check-ins, contradiction/blocker messages
B	GitHub pipeline, verification, commit polling, contribution stats
C	Backend + DB + planner (bundling/assignment) + Guide agent + all tools
D	Frontend (board, plan, graph, contributions, chat)

Timeline (from locked plan)

- Hour 0–1 (all four): agree on DB schema + API contract (this document). Build against mocks.

- Hours 1–8: each builds their piece in isolation.
- Hour 8 checkpoint: core loop works end-to-end (task → ping → reply → status on board).
- Hours 8–16: GitHub verification, Guide, bundling/assignment, graph & charts.
- Hour 16 checkpoint: full flow works once start-to-finish.
- Hours 16–21: contradiction, blocker routing, ripple, messy-reply testing, bug fixes.
- Hours 21–24: README, seeded demo, rehearsals. **No new features after hour 21.**

20. Acceptance checklist (what must work for the live demo)

Judges score the 15 points against these concrete behaviours. Everything below must be demonstrable live.

Core loop (Functional Completeness – 5 pts)

- [ ] Lead creates project via `/newproject <idea>` and receives repo URL + join link.
- [ ] Members join via deep link, supply GitHub username, receive collaborator invite.
- [ ] `/plan` or web button generates 12–20 tasks with criteria and dependencies.
- [ ] Lead can edit/add/delete tasks on the web Plan page.
- [ ] Bundle & Assign produces independent blocks and even loads; oversize blocks flagged.
- [ ] Kickoff sends each member their tasks + deadlines + commit rule on Telegram.
- [ ] Scheduler (or “Run follow-up now”) sends a real Telegram check-in.
- [ ] Member replies with free text (including messy / multi-task replies); status updates correctly.
- [ ] Claiming “done” moves task to claimed_done and triggers GitHub verification.
- [ ] Verification returns confidence + reason; board shows needs_review or verified_done accordingly.
- [ ] Lead can override verification from the web drawer.

Quality of execution (5 pts)

- [ ] Unclear reply produces exactly one clarifying question, no status change.
- [ ] Zero commits → no_evidence verdict without calling the LLM.
- [ ] Commits with wrong author are detected and credited correctly.
- [ ] Re-verification works when a new matching commit arrives.
- [ ] All state changes appear in the activity log within 2 seconds of the poll.
- [ ] Bot never goes silent on exceptions.

Creativity & depth (5 pts)

- [ ] Contradiction: two conflicting statements are detected and both members are messaged.
- [ ] Blocker routing: system chooses a helper, messages them, and updates status when resolved.
- [ ] Ripple: blocking one task correctly marks downstream tasks / milestone as at_risk or critical; graph edges glow.
- [ ] Guide answers at least three of: “Who contributed what?”, “What was completed by [time]?”, “Why is the demo at risk?” citing real task keys / SHAs.
- [ ] Contribution chart shows multi-colored lines (one per member) and a totals table.
- [ ] Final `/report` produces a coherent summary of claimed vs verified, blockers, and risks.

Demo hygiene

- [ ] DEMO_MODE compresses time so the whole loop is visible in <15 minutes.
- [ ] Seeded backup project exists and can be loaded if live GitHub creation fails.
- [ ] README contains exact run instructions and the pitch line.

If all boxes above are checked, the team has a complete, judge-ready product.

End of PRD v1.0 — complete. Any further decisions go in `/docs/decisions.md`.